

APLICACIONES MOVILES

LABORATORIO 3C

La misma app, tres veces: la pantalla se describe

Módulo 2 — Desarrollo nativo · Guía autónoma · Tiempo estimado: 90 minutos · Tercera parte de tres (3A → 3B → 3C)

El problema de hoy: tercera vez, última forma: la misma AppTeca, pero la pantalla ya no se infla ni se muta — se describe. Vas a escribir qué se ve para cada estado, y el redibujado va a dejar de ser tu problema.

Qué necesitás antes de empezar: los proyectos del 3A y 3B a mano (vas a copiar archivos de ahí, y la comparación es el corazón de la trilogía). La ficha comparativa con sus columnas A y B llenas — hoy se completa. Este lab enseña **lo mínimo de Compose para reconstruir AppTeca**: el estudio en profundidad (modificadores, theming, layouts) es materia de la guía de Compose y de la teoría — acá se viene a sentir el contraste.

Teoría que acompaña: el primer eje del bloque de la evolución de las formas de desarrollar — imperativo → declarativo, "la interfaz es una función del estado", la recomposición. Hoy esas frases dejan de ser frases.

El destino

Al terminar vas a tener la tercera AppTeca: idéntica para el usuario (lista, detalle, estrellas, buscador, modo "solo favoritas"), sin un solo layout XML, sin adaptador, sin `findViewById` — la interfaz entera descrita en funciones Kotlin que se redibujan solas cuando el estado cambia. Tu ViewModel del 3B va a mudarse casi sin tocarlo. La grieta que sobrevivió a dos laboratorios va a morir sin que escribas una línea para matarla. Y tu ficha va a quedar completa: tres columnas, tres formas, tu experiencia.

Etapa 1 — El proyecto Compose: cerrar un círculo

En el Laboratorio 1 te hicimos esquivar una plantilla "que ibas a conocer en su momento". El momento llegó.

1.1 — Creá el proyecto. New Project → Empty Activity — esta vez sí, la primera de la galería, la que Android Studio destaca: la plantilla Compose. Name: `AppTeca3` · Minimum SDK: API 26. Sincronizá (la primera sincronización de un proyecto Compose descarga más que de costumbre — mate).

✓ *Verificación: el proyecto abre sin errores y —hacé el recorrido del Lab 1 al revés— no existe `res/layout`. Buscala: no está. La carpeta `res` sigue ahí (strings, colores, íconos: los recursos no se fueron a ningún lado), pero la estructura de las pantallas ya no vive en XML. ¿Dónde vive? Abrí `MainActivity.kt`.*

1.2 — Leé lo que la plantilla trajo. En `MainActivity` no hay `setContentView(R.layout...)`: hay `setContent { ... }` — y adentro, funciones. Fijate la forma de lo que la plantilla generó (los nombres exactos varían con la versión):

```
class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContent {
            AppTeca3Theme {
                Greeting(name = "Android")
            }
        }
    }
}
```

```
@Composable
fun Greeting(name: String, modifier: Modifier = Modifier) {
    Text(text = "Hola $name", modifier = modifier)
}
```

Las piezas nuevas, en tres definiciones. Un **composable** es una función anotada con `@Composable` que *describe* un pedazo de interfaz — no devuelve una vista ni muta nada: declara "acá va esto". `setContent` reemplaza al par XML+inflado: el contenido de la pantalla es directamente una llamada a composables. Y el **Theme** que envuelve todo es Material Design como función — colores, tipografía y formas que los composables de adentro heredan. No hay ids, no hay `findViewById`, no hay dos archivos que mantener de acuerdo: la estructura y el contenido son el mismo código.

1.3 — Tus datos viajan. Copiá desde el proyecto del 3B los archivos `App.kt` y `Catalogo.kt` (tal cual — ajustá solo la línea `package` al paquete nuevo). Son la prueba silenciosa de una idea de la teoría: el modelo de datos no sabe en qué se dibuja.

✓ Verificación: *compila; tus 8 apps están en el proyecto nuevo.*

🔧 Problemas frecuentes de la Etapa 1

- **El preview no renderiza** ("Render problem"): el preview es cómodo pero no es el criterio — el criterio es ejecutar. Si el emulador muestra la app, seguí; el preview a veces necesita un Build → Rebuild.
- **"@Composable invocations can only happen from..."**: llamaste un composable desde una función común. Los composables solo se llaman desde otros composables (o desde `setContent`) — es la regla estructural del modelo, y este error te la va a recordar mil veces.
- **Errores al pegar `App.kt` / `Catalogo.kt`**: la línea `package` tiene que ser la del proyecto nuevo (fijate cómo se llama el paquete en `MainActivity` y usá el mismo).

Etapa 2 — La lista: la ceremonia que no vas a pagar

En el 3A, mostrar la lista costó una etapa entera y cuatro piezas. Contá lo que cuesta acá.

2.1 — La fila. En `MainActivity.kt` (después vas a poder ordenar en archivos — hoy, todo junto para verlo junto), agregá el composable de una fila:

```
@Composable
fun FilaApp(
    app: App,
    onClick: () -> Unit,
    onFavoritoClick: () -> Unit
) {
    Row(
        modifier = Modifier
            .fillMaxWidth()
            .clickable { onClick() }
            .padding(16.dp),
        verticalAlignment = Alignment.CenterVertically
    ) {
        Column(modifier = Modifier.weight(1f)) {
            Text(app.nombre, style = MaterialTheme.typography.titleMedium)
            Text(app.categoria, style = MaterialTheme.typography.bodySmall)
        }
        Text(
```

```

        text = if (app.esFavorita) "★" else "☆",
        fontSize = 24.sp,
        modifier = Modifier
            .clickable { onFavoritoClick() }
            .padding(8.dp)
    )
}
}

```

Leelo contra tu `item_app.xml` + `ViewHolder` del 3A: es **la misma fila** — `Row` por el `LinearLayout` horizontal, `Column` con `weight` por el vertical que se estira, dos `Text` y la estrella. Pero acá estructura, datos y comportamiento son *una sola cosa*: la fila recibe su app y sus lambdas, y describe cómo se ve — no hay ids porque no hay nadie que tenga que salir a buscar vistas: los datos ya están adentro de la descripción. (Los `Modifier` encadenados —tamaño, click, padding— son el sistema de propiedades de Compose; la guía los desarrolla, acá se usan.)

2.2 — La lista. Y ahora el momento que la trilogía preparó. En el 3A: layout del ítem + `ViewHolder` + adaptador con tres métodos + enganche con `layoutManager` — cuatro piezas, dos archivos, ~60 líneas. Acá:

```

@Composable
fun ListaApps(
    apps: List<App>,
    onAppClick: (App) -> Unit,
    onFavoritoClick: (App) -> Unit
) {
    LazyColumn {
        items(apps, key = { it.id }) { app ->
            FilaApp(
                app = app,
                onClick = { onAppClick(app) },
                onFavoritoClick = { onFavoritoClick(app) }
            )
        }
    }
}

```

Eso es todo. `LazyColumn` es el `RecyclerView` de Compose — el mismo reciclado eficiente (solo compone lo visible), la misma preocupación por el presupuesto de recursos, sin adaptador: la función `items` recibe la lista y la descripción de cada fila, y el resto es problema del framework. El `key = { it.id }` le dice quién es quién entre recomposiciones (tu `id` del 3A, trabajando de nuevo). Conectala provisoriamente para verla — en `setContent`, reemplazá el `Greeting` por:

```

AppTeca3Theme {
    ListaApps(
        apps = Catalogo.apps,
        onAppClick = { },
        onFavoritoClick = { }
    )
}

```

Ejecutá.

✓ Verificación: tus 8 apps en pantalla, scrolleables. Anotá para la ficha, pregunta 1: la lista del 3A contra la de hoy — contá piezas y líneas. Esta diferencia no es un truco del ejemplo: es la ceremonia estructural del modelo viejo que el modelo nuevo no necesita.

🔧 Problemas frecuentes de la Etapa 2

- **Imports por todos lados en rojo:** Compose usa muchos — Alt+Enter en cada uno, o dejá que el editor los agregue al pegar. Los conflictivos: `items` es el de `androidx.compose.foundation.lazy.items` (hay varios con ese nombre).
- **"Unresolved reference: dp / sp":** imports de `androidx.compose.ui.unit.dp` y `.sp`.
- **La estrella no queda a la derecha:** el `weight(1f)` de la `Column` es lo que empuja — sin él, nada se estira y todo queda apretado a la izquierda.

Etapa 3 — El estado entra: tu ViewModel se muda, la pantalla se suscribe

El 3B te prometió que el ViewModel se mudaba casi sin tocarlo. Hora de cobrar esa promesa — con un cambio de dos líneas que tiene nombre propio.

3.1 — Las dependencias. En `build.gradle.kts` (`Module :app`), al bloque `dependencies` (la plantilla Compose ya trae casi todo; sumá estas y sincronizá):

```
implementation("androidx.lifecycle:lifecycle-viewmodel-compose:2.8.7")
implementation("androidx.lifecycle:lifecycle-runtime-compose:2.8.7")
```

[⚠ VERIFICAR: versiones vigentes al preparar el dictado]

3.2 — El ViewModel, mudado con un cambio. Copiá tu `AppTecaViewModel.kt` del 3B al proyecto nuevo (paquete ajustado) y hacé el cambio: donde había `LiveData`, va **`StateFlow`**. La clase completa, con el diff señalado:

```
package com.example.appteca3

import android.util.Log
import androidx.lifecycle.ViewModel
import kotlinx.coroutines.flow.MutableStateFlow
import kotlinx.coroutines.flow.StateFlow

class AppTecaViewModel : ViewModel() {

    private var query = ""
    private var soloFavoritas = false

    // — ANTES (3B): MutableLiveData<List<App>>() / LiveData<List<App>>
    // — AHORA:
    private val _listaVisible = MutableStateFlow<List<App>>(emptyList())
    val listaVisible: StateFlow<List<App>> = _listaVisible

    private val _modoSoloFavoritas = MutableStateFlow(false)
    val modoSoloFavoritas: StateFlow<Boolean> = _modoSoloFavoritas

    init {
        Log.d("VIDA", "ViewModel → creado (${hashCode()})")
        aplicarFiltros()
    }

    fun buscar(texto: String) {
        query = texto.trim()
        aplicarFiltros()
    }
}
```

```

fun alternarModo() {
    soloFavoritas = !soloFavoritas
    aplicarFiltros()
}

fun alternarFavorita(app: App) {
    val nuevas = Catalogo.apps.map {
        if (it.id == app.id) it.copy(esFavorita = !it.esFavorita) else it
    }
    Catalogo.apps.clear()
    Catalogo.apps.addAll(nuevas)
    aplicarFiltros()
}

private fun aplicarFiltros() {
    var lista: List<App> = Catalogo.apps.toList()
    if (query.isNotEmpty()) lista = lista.filter {
        it.nombre.contains(query, true) || it.categoria.contains(query,
true)
    }
    if (soloFavoritas) lista = lista.filter { it.esFavorita }
    _listaVisible.value = lista
    _modoSoloFavoritas.value = soloFavoritas
}

override fun onCleared() {
    Log.d("VIDA", "ViewModel → onCleared")
}
}

```

Dos cosas para leer. **La chica:** `StateFlow` es la pieza del ecosistema de corrutinas que cumple el rol que en la era Views cumplía `LiveData` — un valor observable que siempre tiene estado actual y entrega el último valor a cada suscriptor nuevo. Es la teoría de los Flows aterrizando: mismo patrón, familia moderna, y la que `Compose` espera. Nota la diferencia menor: `StateFlow` *exige* valor inicial (`emptyList()`) — no existe el "todavía no publiqué nada". **La grande, en `alternarFavorita`:** ya no mutamos `esFavorita` en el lugar — se construye una **lista nueva** donde la app cambiada es una **copia** (`it.copy(...)` — tu data class del Lab 1, pagando dividendos). ¿Por qué? Porque `Compose` decide qué recomponer **comparando** estados: si mutás el objeto adentro de la misma lista, el estado "nuevo" y el "viejo" son el mismo objeto — la comparación dice "nada cambió" y la pantalla no se entera. La promesa del 1.2 del 3A se cumple: **la inmutabilidad dejó de ser estilo y pasó a ser mecánica** — en el mundo declarativo, cambiar es reemplazar. (Sí: el `var esFavorita` del modelo quedó como reliquia del 3A — podés convertirlo en `val`; el compilador te va a señalar solo los lugares viejos que mutaban.)

3.3 — La pantalla completa se suscribe. Ahora el composable raíz de la pantalla — con el buscador y el botón de modo, y la suscripción a los flujos:

```

@Composable
fun PantallaAppTeca(vm: AppTecaViewModel = viewModel()) {
    val lista by vm.listaVisible.collectAsStateWithLifecycle()
    val modoFav by vm.modoSoloFavoritas.collectAsStateWithLifecycle()
    var textoBusqueda by rememberSaveable { mutableStateOf("") }

    Column(modifier = Modifier.fillMaxSize()) {
        OutlinedTextField(
            value = textoBusqueda,
            onChange = { nuevo ->
                textoBusqueda = nuevo
                vm.buscar(nuevo)
            },

```

```

        label = { Text("Buscar por nombre o categoría...") },
        modifier = Modifier
            .fillMaxWidth()
            .padding(16.dp)
    )

    Button(
        onClick = { vm.alternarModo() },
        modifier = Modifier
            .fillMaxWidth()
            .padding(horizontal = 16.dp)
    ) {
        Text(if (modoFav) "★ Solo favoritas" else "☆ Todas")
    }

    ListaApps(
        apps = lista,
        onAppClick = { /* Etapa 4 */ },
        onFavoritoClick = { app -> vm.alternarFavorita(app) }
    )
}

```

Y en `setContent: AppTeca3Theme { PantallaAppTeca() }`. (Imports: `viewModel` de `androidx.lifecycle.viewmodel.compose`, `collectAsStateWithLifecycle` de `androidx.lifecycle.compose`.) Las piezas nuevas, leídas de arriba a abajo: **`collectAsStateWithLifecycle()`** es el `observe` de esta era — convierte el `StateFlow` en estado que Compose observa, con la misma conciencia de ciclo de vida que `LiveData` tenía (no colecta con la pantalla invisible); el `by` hace que `lista` se lea como variable común, y acá está el corazón de todo: **cuando el flujo publica, `lista` cambia — y toda función que la lee se recompone sola**. No hay `adapter.actualizarLista`, no hay suscripción que escribir con qué-hacer-cuando-llegue: leer el estado es suscribirse. El campo de texto muestra el otro gesto del modelo — el **componente controlado**: el `OutlinedTextField` no guarda su texto (no hay vista con estado propio que el sistema salve, como el `EditText` del 3A); su valor es `textoBusqueda`, y escribir dispara `onValueChange`, que actualiza el estado, que recompone el campo con el carácter nuevo. El círculo completo en cada tecla — estado abajo, eventos arriba, sin excepciones. (`rememberSaveable` protege ese texto de la recreación — estado efímero *de UI* que no amerita `ViewModel`; el matiz fino queda para la guía.)

Ejecutá y usá todo: buscá, alterná el modo, marcá estrellas.

✓ *Verificación: todo funciona — y repasá lo que NO escribiste: ninguna llamada de redibujado, ningún `notify`, ningún `observe` con lambda de actualización. Escribiste qué se ve para cada estado; el cuándo redibujar dejó de existir como pregunta. "La interfaz es una función del estado" — acabás de ejecutarla, literalmente.*

🔧 Problemas frecuentes de la Etapa 3

- **La estrella no se actualiza al tocarla:** volviste a mutar en el lugar (`app.esFavorita = ...`) en vez de reemplazar con copias — releé "la grande" del 3.2: para Compose, mutar adentro es invisible.
- **"Unresolved reference: `collectAsStateWithLifecycle`":** falta la dependencia `lifecycle-runtime-compose` o su import exacto.

- **El teclado se cierra o el cursor salta al escribir:** síntoma clásico de componente controlado mal cableado — verificá que `value` lea el estado y `onValueChange` lo actualice *siempre* (si a veces no lo actualizás, el campo "pelea" con el usuario).
- `viewModel()` **no resuelve:** es el de `androidx.lifecycle.viewmodel.compose.viewModel` — y cumple el rol del `by viewModel()` del 3B: pedirle la instancia al sistema, no construirla.

Etapa 4 — La navegación como estado — y la muerte de la grieta

En 3A y 3B, "navegar" era pedirle al sistema otra Activity. Acá va a ser cambiar un valor — y un bug que sobrevivió dos laboratorios va a morir sin funeral.

4.1 — El estado de navegación. En el ViewModel, tres agregados chicos:

```
private val _appSeleccionada = MutableStateFlow<App?>(null)
val appSeleccionada: StateFlow<App?> = _appSeleccionada

fun seleccionar(app: App) { _appSeleccionada.value = app }
fun volverALista() { _appSeleccionada.value = null }
```

Y una línea al final de `aplicarFiltros()`, para que el detalle también respire del estado vivo:

```
_appSeleccionada.value = _appSeleccionada.value?.let { sel ->
    Catalogo.apps.find { it.id == sel.id }
}
```

Leé el tipo: `App?` — **la pantalla actual es un valor nutable.** Hay app seleccionada → se está viendo el detalle; null → la lista. La navegación dejó de ser un mecanismo (Intents, pila, extras) y pasó a ser un dato más del estado.

4.2 — La pantalla de detalle. Un composable nuevo — compará con tu `DetalleActivity` + `activity_detalle.xml` mientras lo escribís:

```
@Composable
fun DetalleApp(app: App, onFavoritoClick: () -> Unit, onVolver: () -> Unit) {
    BackHandler { onVolver() }
    Column(modifier = Modifier.fillMaxSize().padding(24.dp)) {
        Text(app.nombre, style = MaterialTheme.typography.headlineLarge)
        Text(app.categoria, style = MaterialTheme.typography.bodyMedium)
        Spacer(Modifier.height(16.dp))
        Text(app.descripcion, style = MaterialTheme.typography.bodyLarge)
        Spacer(Modifier.height(24.dp))
        Button(onClick = onFavoritoClick) {
            Text(if (app.esFavorita) "★ Quitar de favoritas" else "★ Marcar favorita")
        }
    }
}
```

`BackHandler` es la pieza que completa el modelo: intercepta el gesto/botón de atrás y lo convierte en lo que "atrás" significa en este diseño — deseleccionar. Sin él, atrás cerraría la app entera (no hay pila de Activities que desapilar: hay una sola).

4.3 — El `if` que es toda la navegación. En `PantallaAppTeca`, sumá la suscripción y el desvío — el `Column` entero queda envuelto así:

```
val seleccionada by vm.appSeleccionada.collectAsStateWithLifecycle()

if (seleccionada != null) {
    DetalleApp(
        app = seleccionada!!,
        onFavoritoClick = { vm.alternarFavorita(seleccionada!!) },
        onVolver = { vm.volverALista() }
    )
} else {
    Column(modifier = Modifier.fillMaxSize()) {
        // ... el buscador, el botón y la lista, tal como estaban ...
    }
}
```

Y el hueco pendiente de la Etapa 3 se llena: `onAppClick = { app -> vm.seleccionar(app) }`. Ejecutá: tocá una app → detalle; atrás → lista, con búsqueda y modo intactos (¿viste? — el estado de la lista ni se enteró del viaje).

4.4 — El experimento de la grieta — por tercera vez. El ritual completo, idéntico a 3A y 3B: entrá al detalle de una app **sin** estrella, marcala favorita con el botón, volvé.

✓ Verificación: la fila en la lista muestra ★ — actualizada. Sin `onResume`, sin `refrescar()`, sin parche: la grieta murió. ¿Por qué acá sí? Repasá la anatomía: en 3A/3B eran dos *Activities* — dos mundos, cada uno con su foto de los datos, y la sincronización entre mundos era artesanal. Acá el detalle y la lista son dos descripciones derivadas del mismo estado del mismo *ViewModel*: `alternarFavorita` publica la lista nueva, y todo lo que la lee —la fila, el botón del detalle— se recompone. No arreglaste el bug: elegiste una arquitectura donde ese bug no tiene dónde existir. (La honestidad de siempre: esto funcionó porque ambas "pantallas" comparten *ViewModel*. En apps reales con la biblioteca de navegación y *ViewModels* por pantalla, la fuente de verdad compartida vuelve a ser un problema de diseño — con solución en la capa de datos, como anticipó el 3B. La biblioteca y ese diseño son territorio de la guía y del TP.)

🔧 Problemas frecuentes de la Etapa 4

- **Atrás cierra la app desde el detalle:** falta el `BackHandler` (import de `androidx.activity.compose.BackHandler`) — sin él no hay quien traduzca "atrás" a "deseleccionar".
- **El botón del detalle cambia pero la estrella de la lista no** (o viceversa): alguna de las dos descripciones no está leyendo del estado colectado — buscá si quedó alguna referencia vieja a un objeto suelto en vez del flujo.
- **"Smart cast to 'App' is impossible":** el compilador desconfía del `seleccionada` nutable entre lecturas — el `!!` del listado lo fuerza (en el TP vas a ver la forma prolija con una variable local: `val app = seleccionada; if (app != null) { ... }`).

Etapa 5 — La prueba de fuego final — y la ficha completa

Tercera vez el mismo ritual. Ya no es para descubrir nada: es para medir con la misma vara — y cerrar la trilogía con tu ficha llena.

5.1 — El giro, por última vez. Marcá favoritas, escribí en el buscador, activá el modo, **entrá al detalle de una app** — y rotá ahí, en el detalle.

✓ *Verificación: todo en su lugar — incluso la pantalla en la que estabas. El detalle sigue abierto (la selección era estado del ViewModel: sobrevivió), y al volver, búsqueda, modo y estrellas intactas (ViewModel + `rememberSaveable`, cada uno cuidando lo suyo). Fijate el detalle fino comparando con el 3B: allá el detalle sobrevivía al giro porque el sistema reentregaba el Intent (Lab 2); acá sobrevive porque la selección es estado como cualquier otro. Dos mecanismos distintos, misma experiencia — y el de hoy no necesitó que el sistema hiciera magia: solo que el estado viviera en el lugar correcto.*

5.2 — La muerte de proceso — y un detalle para ojos entrenados. El ritual completo: favoritas, búsqueda escrita, modo activo, Home, `adb shell am kill com.example.appteca3`, volver desde recientes.

✓ *Verificación: favoritos y modo, en blanco — el ViewModel y el singleton murieron con el proceso, igual que en el 3B: Compose no cambió nada de esto, porque esto nunca fue de la capa de interfaz. Pero mirá fino, con lo que ya sabés: el texto de búsqueda volvió (`rememberSaveable` usa el mecanismo de rescate del sistema — el mismo que salvaba al EditText del 3A — y ese sobrevive a la muerte de proceso)... y sin embargo la lista está sin filtrar: el texto se restauró sin disparar `onValueChange`, y el `query` del ViewModel nació vacío. La incoherencia del 3A, reapareciendo en el borde más extremo — porque las dos defensas quedaron desalineadas: una pieza del estado usó el rescate del sistema y la otra no. La cura es alinearlas (el `SavedStateHandle` del desafío del 3B, ahora con motivo propio — desafío 3 de hoy) o que los datos vivan en persistencia real. Anotalo: hasta en el modelo declarativo, el mapa de qué-sobrevive-a-qué sigue siendo responsabilidad del que diseña.*

5.3 — La ficha, completa — y su lectura. Llená la columna C. Y ahora hacé lo que la trilogía preparó: **leé las tres columnas juntas**, fila por fila. Tu ficha cuenta, con tu experiencia como evidencia, exactamente la historia del bloque de teoría: la fila 1 mide la ceremonia (cuatro piezas → cuatro piezas → una función); las filas 2 y 3 miden dónde vive el estado y qué le pasa (variables que mueren → ViewModel que sobrevive → lo mismo, más el estado de navegación); la fila 5 mide la testeabilidad (imposible → de regalo → de regalo); la fila 6 es tuya. Con la ficha en la mano, respondé por escrito las dos preguntas de cierre — van a reaparecer en la defensa del TP y en el parcial:

1. Un equipo mantiene una app Views+XML grande, estable y rentable. ¿Le recomendás reescribirla en Compose? ¿Qué le recomendás, y con qué de tu ficha lo fundamentás?
2. El mismo equipo arranca un producto nuevo desde cero. ¿Qué stack elegís y por qué — citando al menos tres filas de tu ficha?

✓ *Verificación: ficha completa, dos respuestas escritas. Si en la primera respondiste "reescribir", volvé a la fila 4 de tu columna B y pensá qué costó — y qué habría costado con veinte pantallas.*

Desafíos opcionales

Sin guía paso a paso — de eso se trata.

3. **La biblioteca de verdad.** Migrá la navegación a **Navigation Compose**: el NavHost, las rutas, el controlador. Al terminar, compará con el `if` que reemplazaste: ¿qué ganaste (pila real, rutas, transiciones)? ¿qué ceremonia pagaste? ¿en qué tamaño de app se vuelve obligatoria? Es exactamente lo que vas a usar en el TP — este desafío es ventaja pura.

4. **El caramelo.** Agregale `Modifier.animateItem()` a las filas de la `LazyColumn` y filtrá: las filas se acomodan animadas, gratis. Investigá por qué el `key = { it.id }` que ya tenías es lo que lo hace posible.
5. **Alinear las defensas.** Resolvé la incoherencia del 5.2: mudá `query` (y `soloFavoritas`) al `SavedStateHandle` del `ViewModel`, y hacé que el campo de texto sea controlado *desde el ViewModel* (el texto como `StateFlow`, sin `rememberSaveable`). Repetí la muerte de proceso: ¿qué quedó coherente ahora? ¿Qué sigue muriendo — y por qué eso ya no es un problema de estado sino de datos?

Checklist de salida

- ☐ AppTeca3 completa en Compose: lista, detalle, estrellas, buscador y modo — sin XML, sin adaptador, sin `findViewById`
- ☐ Sé explicar por qué `alternarFavorita` reemplaza con copias en vez de mutar — y qué pasa si muto
- ☐ Entiendo el componente controlado (el buscador) y el ciclo estado-abajo/eventos-arriba en cada tecla
- ☐ La navegación es un `App?` con un `if` — y sé qué me daría Navigation Compose a cambio de su ceremonia
- ☐ Vi morir la grieta del detalle y sé explicar por qué esta arquitectura no tiene dónde alojarla
- ☐ Hice la prueba de fuego completa, incluido el detalle fino del texto que vuelve sin su filtro
- ☐ Ficha completa: tres columnas y las dos preguntas de cierre respondidas
- ☐ (Opcional) Navigation Compose · `animateItem` · las defensas alineadas

Adónde va esto — el cierre de la trilogía: construiste la misma app tres veces, y ya no hay slogan que te la cuente: sabés qué cuesta cada forma, qué protege, qué rompe y qué regala — con tu ficha como prueba. El stack del TP queda elegido con fundamento: **Compose + MVVM**, la combinación de tu columna C, que es la que la plataforma recomienda y la industria adopta para lo nuevo — sabiendo, porque lo escribiste en la pregunta de cierre, que la columna A sigue viva en millones de apps que alguien tiene que mantener, y que "no reescribir por moda" también es criterio profesional. Lo que le falta a tu AppTeca para ser el TP es lo que le falta a todo este módulo: que los datos no mueran con el proceso y que vengan de donde vienen los datos reales — **un backend**. Eso es el Laboratorio 4: Firebase, y tu catálogo viviendo en la nube.